Coding & Solution

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

Solution Summary

Repository Link / URL	https://github.com/RamanHarshitha/Smart_Bridge
Programming Language(s)	Python 3.10+, HTML5, CSS3, JavaScript (Vanilla)
Framework(s) Used	No external web framework. Pure Python http.server.BaseHTTPRequestHandler + ThreadingHTTPServer. Scikit-learn for ML. TensorFlow-CPU (optional) for CNN. Chart.js 4.4.1 (CDN) for frontend analytics charts.
Key Features Implemented	1. Crop Recommendation — POST /api/recommend: RF+DT+SVC ensemble (crop_model.joblib), 9 features, 22 crop classes, probability averaging. 2. Crop Suitability Assessment — POST /api/suitability: weighted 7-metric scoring with remediation steps. 3. Fertilizer Recommendation — POST /api/fertilizer: NPK gap analysis with crop-specific nutrient advice and pH soil note. 4. Symptom Disease Detection — POST /api/disease: RF classifier on crop+symptoms+environment (disease_model.joblib). 5. Image Disease Detection — POST /api/image-disease: CNN (Keras .h5) or pixel-feature RF (image_disease_model.joblib) on uploaded leaf image. 6. Real-time Weather Auto-fill — GET /api/weather: Open-Meteo geocoding + forecast API (no key, city or GPS). 7. Research Analytics Dashboard — GET /api/analytics: Chart.js bar/radar/scatter, filter by crop/region/season, CSV export.
Pending / Incomplete Features	Multilingual interface (Telugu, Hindi). Mobile APK build. IoT soil sensor integration. District-level crop recommendation model. Crop yield prediction (regression). Market price API integration.
Setup / Run Instructions	1. git clone https://github.com/RamanHarshitha/Smart_Bridge 2. cd Smart_Bridge-master 3. pip install -r requirements.txt 4. python3 ml/train_model.py (trains crop model — requires Crop_recommendation.csv in data/) 5. python3 ml/train_disease_model.py (trains disease model) 6. python3 server.py 7. Open http://127.0.0.1:8000 in a browser

Code Quality Checklist

S.No	Criteria	Status (Yes / No)

1	Code is modular and organised into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The crop_model.joblib bundle contains three trained scikit-learn models (RandomForestClassifier, DecisionTreeClassifier, SVC via Pipeline), their LabelEncoders, and the feature list — loaded once at server startup and reused across all requests. The server implements a graceful fallback: if the model file is missing, /api/recommend returns a hardcoded fallback response with common crops and a 'fallback: true' flag, ensuring the UI never crashes. The Open-Meteo weather integration requires no API key and works on any internet-connected machine.